JAGUAR: Efficient and Secure Unbalanced PSI under Malicious Adversaries in the Client-Server Setting

Weizhan Jing[1,2,3], Xiaojun Chen[1,2,3*], Xudong Chen[1,3], Ye Dong[4], Qiang Liu[1,2,3], Tingyu Fan[1,2,3]

[1]Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China.
[2]School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China.
[3]State Key Laboratory of Cyberspace Security Defense, Beijing, China.
[4]School of Computing, National University of Singapore, Singapore.

*Corresponding author(s). E-mail(s): chenxiaojun@iie.ac.cn;
Contributing authors: jingweizhan@iie.ac.cn; chenxudong@iie.ac.cn; dongye@nus.edu.sg; liuqinag2022@iie.ac.cn; fantingyu@iie.ac.cn;

Abstract

In many unbalanced private set intersection (uPSI) applications of the client-server setting, the server needs to perform uPSI with multiple clients. Cong *et al.* (ACM CCS'21) proposed a state-of-the-art (SOTA) uPSI protocol based on fully homomorphic encryption (FHE), achieving malicious security by employing an oblivious pseudorandom function (OPRF) in the pre-processing phase. However, re-executing existing uPSI protocols with each client imposes significant computational overhead for the server. In this paper, we present JAGUAR, a maliciously secure and efficient uPSI protocol designed for this setting. JAGUAR reduces online computation through a Divide-and-Combine optimization, requiring only $\mathcal{O}(\sqrt{|X|})$ homomorphic multiplications. Furthermore, it employs a novel fixed VOLE-based OPRF that enables reusable and lightweight pre-processing across multiple clients. Experimental results demonstrate that JAGUAR achieves up to **2.7**× improvement in online runtime compared to the SOTA protocol in LAN. In multi-client scenarios, JAGUAR further outperforms existing protocols by a wide margin in terms of scalability and overall performance.

Keywords: Unbalanced Private Set Intersection, Vector-OLE, Fully Homomorphic Encryption, Oblivious Pseudo-Random Functions.

1 Introduction

The two-party private set intersection (PSI) is a secure protocol that enables a server and a client, each with their private sets X and Y, to compute $X \cap Y$ without revealing any additional information. The PSI protocol serves as an important primitive in various privacy-preserving protocols, such as private genetic testing (Baldi et al. 2011; Ghosh and Simkin 2019), private contact discovery (Kiss et al. 2017; Kales et al. 2019; Wu and Yuen 2023; Hetz et al. 2023; Chen et al. 2017, 2024), private ID alignment (He et al. 2022), and private suspect-list tests (Kissner and Song 2004). In many client-server scenarios, multiple clients with small private datasets often need to securely compute the intersection with a significantly larger dataset stored by the cloud server

(a.k.a. unbalanced case). For instance, in a private contact discovery service, the server maintains a large database of all users, while each mobile client holds a relatively small list of personal contacts and aims to determine which of their contacts are also users of the application.

Theoretically, any secure PSI protocol can be applied in the unbalanced case. Over the last several years, many efficient PSI protocols (Pinkas et al. 2014, 2015; Kolesnikov et al. 2016; Pinkas et al. 2018b, 2019b, 2020; Rindal and Schoppmann 2021; Raghuraman and Rindal 2022) have been developed. Among them, the work of Raghuraman and Rindal (2022) achieves the most efficient protocol. However, the communication overhead[1], which is linear with the set sizes of both parties, limits its applicability in unbalanced client-server settings.

Recently, several PSI protocols (Chen et al. 2017; Kiss et al. 2017; Chen et al. 2018; Resende and Aranha 2018; Kales et al. 2019; Resende and Aranha 2021; Cong et al. 2021) have been developed for the unbalanced case. Among these works, Kiss et al. (2017), Resende and Aranha (2018) and Kales et al. (2019) utilize the oblivious pseudorandom function (OPRF) and the cuckoo filter (CF) to construct efficient PSI protocols that achieve $\mathcal{O}(|Y|)$ computational and communication complexity in the online phase. Unfortunately, the CF incurs a non-negligible false-positive rate (FPR). In the experiments of Resende and Aranha (2018) and Kales et al. (2019) (abbreviated as KRS+19 hereafter), they parameterized the cuckoo filter to have a FPR of 2^{-13} and 2^{-29}, respectively.

Instead, the fully homomorphic encryption (FHE)-based protocols (Chen et al. 2017, 2018; Cong et al. 2021) do not suffer from FPR problems and Cong et al. (2021) (abbreviated as CMDG+21 hereafter) achieves the state-of-the-art (SOTA).

1.1 FHE-based PSI Protocol

We provide an overview of the CMDG+21 as follows. The core idea is bin-wise matching.

The client first performs the cuckoo hashing to insert the smaller set Y into a cuckoo hash table CH with bin size one. Meanwhile, the server inserts the larger set X into the simple hash table SH with bin size B. After the hashing, the client only needs to compare each item y in CH with the items $x_1, x_2, ..., x_B$ in SH at the same bin location. Then, the server constructs the polynomial $P(x)$ for each hash bin.

$$P(x) = \prod_{i=1}^{B}(x - x_i) = a_0 + a_1 \cdot x + \cdots + a_B \cdot x^B$$

where $a_0, a_1, \cdots, a_B$ are the coefficients. Then, the client encrypts the item y and sends it to the server. For an encryption $[\![y]\!]$, the server homomorphically computes the polynomial

$$[\![z]\!] := rP([\![y]\!]) = r \cdot (a_0 + a_1 \cdot [\![y]\!] + \cdots + a_B \cdot [\![y^B]\!])$$

where $r \in_R \mathbb{F}$. Then, the client decrypts the $[\![z]\!]$, received from the server, to check whether $z = 0$ and outputs y as an intersection item if it is true.

Building on this general protocol, previous works (Chen et al. 2017, 2018; Cong et al. 2021) proposed several optimizations to improve the protocol efficiency and achieve $\mathcal{O}(|Y|\log(|X|))$ communication complexity and $\mathcal{O}(\sqrt{|X|})$ homomorphic multiplications computation complexity. However, from a practical perspective, employing the SOTA FHE-based PSI in the client-server setting may be prohibitive for two main reasons:

i) **Inefficiency in Large-Scale Polynomial Evaluation**. Although the asymptotic complexity of homomorphic multiplication can be reduced to sublinear by the optimizations proposed in Chen et al. (2017, 2018); Cong et al. (2021), this cost remains significant in large-scale client-server scenarios, where the server's dataset may contain millions or even billions of items. In such settings, the homomorphic multiplications continue to dominate the online runtime, imposing a substantial computational burden on the server. For example, as reported in CMDG+21, intersecting 2^{20} (resp. 2^{24}) items with a client set of size 11041 requires 42.4 (resp. 59.6) seconds for the server to process queries from four clients. This demonstrates that, despite having sublinear complexity, the concrete performance cost can still be a major bottleneck in practice.

ii) **Non-reusable OPRF Phase**. The work of CMDG+21 uses the Diffie-Hellman (DH) or oblivious transfer (OT)-based OPRF preprocessing

[1]The communication overhead refers to the total amount of data transmitted between the participating parties during the execution of the protocol.

phase to achieve malicious security. In the OPRF phase, the server holds the secret key k and computes the PRF result X' locally. In contrast, for the set Y of the client, two parties interactively perform the OPRF to map Y to the pseudorandom set Y'. However, since both OPRF methods are non-reusable for the client, the server must perform the OPRF repeatedly for each client, which hinders the scalability of PSI protocols in the client-server scenario.

1.2 Our solutions

In this paper, we focus on improving the computational efficiency of the FHE-based PSI protocol to make it more suitable for the client-server scenario. To tackle the above problems, we propose JAGUAR, an efficient PSI protocol from FHE and vector oblivious linear-function evaluation (VOLE) with two novel approaches: Divide-and-Combine (DaC) and Fixed VOLE-based OPRF (FV-OPRF).

First, we propose the DaC method to reduce the number of homomorphic multiplications in polynomial evaluation, thereby significantly reducing the computational overhead of the protocol. With the partitioning method proposed in Chen et al. (2017), each bin is sliced into α sub-bins, and each sub-bin generates α polynomial evaluation results. To reduce the degree of each polynomial—and consequently the number of homomorphic multiplications—we further divide each sub-bin into β tiny bins, which reduces the homomorphic multiplications from $B/\alpha - 1$ to $B/(\alpha\beta) - 1$. However, this fine-grained partitioning increases the number of intermediate evaluation results to $\alpha\beta$. To avoid a corresponding increase in communication cost, we introduce an additional aggregation step: each group of β polynomial results is homomorphically combined into a single ciphertext. As a result, DaC reduces the total number of homomorphic multiplications by approximately $B/\alpha - B/(\alpha\beta) - \alpha(\beta - 1)$. Since B (the bin size) is typically large in practice while α and β remain relatively small, this reduction is substantial.

Second, we propose the FV-OPRF to enable a lightweight and reusable preprocess phase. We first propose a malicious fixed VOLE protocol, which could generate s VOLE instances ($\boldsymbol{c}_j = \Delta_j \cdot \boldsymbol{a} + \boldsymbol{b}_j, j \in [s]$) with the fixed vector $\boldsymbol{a}$. Then, we apply the malicious fixed VOLE in the VOLE-based OPRF. Concretely, in the FV-OPRF, the server and multiple clients invoke the fixed VOLE protocol to generate tuples $\boldsymbol{c}_i = \Delta_i \cdot \boldsymbol{a} + \boldsymbol{b}_i$, where the server holds $\boldsymbol{c}_i, \boldsymbol{a}, i \in [s]$ and the i-th client holds $\Delta_i, \boldsymbol{b}_i$. Then, the server encodes its own private set X into a vector $\boldsymbol{p}$, masks it by $\boldsymbol{a}$, and publishes $\boldsymbol{a}' = \boldsymbol{p} + \boldsymbol{a}$ to all the clients, who use the vector $\boldsymbol{a}'$ along with the $\Delta_i, \boldsymbol{b}_i$ to compute the PRF result of set Y_i. Since the vector $\boldsymbol{a}$ is fixed in multiple VOLE instances, all clients can reuse one encoding result $\boldsymbol{a}'$ to compute the PRF result of their own set Y_i. Therefore, FV-OPRF solves the problem that the server needs to repeat the OPRF process with each client and improves the protocol efficiency.

1.3 Contributions

In summary, we have the following contributions:

- We propose the DaC optimization, which reduces the number of homomorphic multiplications required in polynomial evaluation, thereby improving computational cost without increasing the communication overhead. Compared to the works of CMDG+21, JAGUAR reduces the online runtime by up to 2.7× (2 × on average) in the LAN setting.
- We propose the FV-OPRF to enable a lightweight and reusable preprocessing phase. Compared to the protocol of CMDG+21, JAGUAR improves the offline runtime by 3× on average.
- We implement and compare JAGUAR with other uPSI in the client-server setting. JAGUAR offers significant advantages in terms of runtime and communication overhead. For 32 clients, JAGUAR is at least 1.8× and 4× faster than CMDG+21 and KRS+19 in terms of total runtime in the LAN setting, respectively.

1.4 Organization

The remainder of this paper is organized as follows. In section 2, we introduce the notation and review the necessary preliminaries. We present our two core optimizations—the DaC technique and the FV-OPRF construction—in section 3 and section 4, respectively. The complete JAGUAR

protocol is described in section 5. Experimental results and performance comparisons are discussed in section 6. We review related work in section 7 and conclude the paper in section 8.

2 PRELIMINARIES

2.1 Notations

We use X, Y to denote sets owned by the server and the client. $[n]$ denotes a set $\{1, ..., n\}$. d is the length of the vector generated by VOLE. we use $\boldsymbol{a}$ to denote the row vector and $\boldsymbol{a}[i]$ is the element indexed by i. For a matrix M, M_i denotes its i-th row vector, and $M_i[j]$ denotes the element at row i and column j. CH and SH denote the hash tables of the cuckoo hashing and simple hashing, respectively. CH_i (or SH_i) is a bin of the hash table indexed by i. $[\![\cdot]\!]$ indicates the ciphertext. $r \leftarrow \mathbb{F}$ means that r is assigned a uniformly random element from the field $\mathbb{F}$. Using κ, λ to denote the computational and statistical security parameters.

2.2 Leveled Fully Homomorphic Encryption

Fully homomorphic encryption (FHE) (Gentry 2009; Fan and Vercauteren 2012) is a special HE scheme that allows an unlimited number of evaluations on encrypted data while preserving the features of the function and format of the encrypted data. The evaluation result remains encrypted and can be recovered by the party holding the decryption key. Many existing FHE schemes are based on learning with errors (LWE) (Regev 2009) or its ring variant (RLWE) (Lyubashevsky et al. 2010) problems. However, its implementation in the real world comes with a huge cost.

To reduce the cost, *leveled*-FHE (Brakerski et al. 2014), whose security parameters are chosen to support circuits only with a certain depth, was introduced for better performance. We provide a high-level overview of the *leveled*-FHE scheme in Figure 1. We use the *leveled*-FHE scheme BFV (Fan and Vercauteren 2012), which has been implemented in the SEAL (SEAL 2022) libraries, in our work to perform the uPSI protocol.

2.3 Vector OLE

Vector oblivious linear-function evaluation (VOLE) (Schoppmann et al. 2019; Weng et al. 2021) is a secure two-party protocol that allows the parties to obtain random vectors $\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c} \in \mathbb{F}^d$ and an element $\Delta \in \mathbb{F}$ such that $\boldsymbol{c} = \Delta \cdot \boldsymbol{a} + \boldsymbol{b}$. The server holds $\boldsymbol{a}, \boldsymbol{c}$, and the client holds $\Delta, \boldsymbol{b}$.

This paper also employs the fixed VOLE (Jing et al. 2025), which can be seen as a variant of the general VOLE, to construct the reusable OPRF for the client. The ideal functionality $\mathcal{F}_{\text{FixedVOLE}}$ of fixed VOLE is shown in Figure 2. Specifically, the $\mathcal{F}_{\text{FixedVOLE}}$ generates n VOLE instances $\boldsymbol{c}_i = \Delta_i \cdot \boldsymbol{a} + \boldsymbol{b}_i, i \in [n]$, wherein the server holds $\boldsymbol{c}_i, \boldsymbol{a}, i \in [n]$ and the i-th client holds $\Delta_i, \boldsymbol{b}_i$, all sharing the same vector $\boldsymbol{a}$.

***Leveled*-FHE scheme**

Public Params: Plaintext modules t, ring dimension n, ciphertext modulus q, and security parameters λ.

- FHE.keyGen(1^λ): Outputs the secret key sk and public key pk of the homomorphic encryption according to the security parameter λ.
- FHE.Enc(m, pk): Encrypts the plaintext $m \in R_t$ with the public key pk and outputs the ciphertext $c \in R_q$.
- FHE.Dec(c, sk): Decrypts the ciphertext $c \in R_q$ with the secret key sk and outputs the plaintext $m \in R_t$.
- FHE.Eva($(c_1, ...c_k), f$): Calculates and outputs the result of the arithmetic circuit f with k input wires over the inputs $c_1, ..., c_k$.

Fig. 1 Algorithms of *leveled*-FHE scheme

2.4 VOLE-based OPRF

Oblivious pseudorandom function (OPRF) (Freedman et al. 2005) is a cryptographic primitive that allows a server to learn a PRF key k and a client to learn the PRF output $F_k(y_1), ..., F_k(y_n)$ on its inputs $y_1, ..., y_n \in Y$. Nothing about the client's inputs is revealed to the server and nothing more about the key k is revealed to the client. Many OPRF constructions (Kolesnikov et al. 2016; Pinkas et al. 2019a; Chase and Miao 2020; Rindal and Schoppmann 2021; Raghuraman and Rindal 2022) have been proposed in recent years

Functionality $\mathcal{F}_{\text{FixedVOLE}}$

Public Params: Two parties, the server ($\mathcal{S}$) and the client ($\mathcal{C}$). A finite field $\mathbb{F}$, the length of VOLE tuples d, the number of VOLE instances n.

Functionality: Upon receiving ($\mathcal{S}$, **sid**) from $\mathcal{S}$ and ($\mathcal{C}$, **sid**) from $\mathcal{C}$,

1) $\mathcal{F}_{\text{FixedVOLE}}$ samples random vectors $\{\boldsymbol{b}_i\}_{i\in[n]} \leftarrow \mathbb{F}^{d\times n}$, $\boldsymbol{a} \leftarrow \mathbb{F}^d$ and integers $\{\Delta_i\}_{i\in[n]} \leftarrow \mathbb{F}^n$. Then, it computes $\boldsymbol{c}_i = \Delta_i \cdot \boldsymbol{a} + \boldsymbol{b}_i$ for $i \in [n]$.
2) $\mathcal{F}_{\text{FixedVOLE}}$ sends $\{\Delta_i\}_{i\in[n]}, \{\boldsymbol{b}_i\}_{i\in[n]}$ to $\mathcal{C}$ and $\boldsymbol{a}, \{\boldsymbol{c}_i\}_{i\in[n]}$ to $\mathcal{S}$.

Fig. 2 Ideal functionality $\mathcal{F}_{\text{FixedVOLE}}$ of fixed VOLE

VOLE-based OPRF

Public Params: The vector length d.

Parties: The server ($\mathcal{S}$) with set X and the client ($\mathcal{C}$) with set Y.

Protocol:

1) Two parties invoke the VOLE protocol, which outputs $\Delta, \boldsymbol{b}$ to $\mathcal{C}$ and $\boldsymbol{a}$, $\boldsymbol{c} = \Delta \cdot \boldsymbol{a} + \boldsymbol{b}$ to $\mathcal{S}$.
2) $\mathcal{S}$ solves the systems

$$M \cdot \mathbf{p} = [H^{\mathbb{F}}(x_1), \ldots, H^{\mathbb{F}}(x_{|X|})]$$

where $M \in \mathbb{F}^{|X|\times d}$ and $M_i = row(x_i)$ s.t. $row(\cdot) : \mathbb{F} \to \mathbb{F}^d$.
3) $\mathcal{S}$ sends $\boldsymbol{a}' = \mathbf{p} + \boldsymbol{a}$ to $\mathcal{C}$, and outputs the set $X' = \{H(\langle row(x), \boldsymbol{c}\rangle) | x \in X\}$.
4) $\mathcal{C}$ sets $\boldsymbol{k} = \boldsymbol{b} + \Delta \cdot \boldsymbol{a}'$, and outputs $Y' = \{H(\langle row(y) \cdot \boldsymbol{k}\rangle - \Delta \cdot H^{\mathbb{F}}(y)) | y \in Y\}$.

Fig. 3 The Protocol of VOLE-based OPRF

and VOLE-based OPRF is one of the most efficient methods. We provide a brief illustration of the VOLE-based OPRF in Figure 3.

2.5 Hashing Technique

Simple hashing. In the simple hashing scheme, a set of n elements is inserted into a hash table SH using w independent hash functions $H_1, \ldots, H_w : \{0,1\}^* \to [m]$, where m denotes the number of bins in SH. Each element x is inserted into w hash bins $SH_{H_1(x)}, \ldots, SH_{H_w(x)}$, regardless of whether these bins are already occupied. According to the analysis in Chen et al. (2017), when inserting $d = wn$ items into m bins in the unbalanced case ($m \ll d$), the maximum bin load in SH is upper-bounded by $d/m + \mathcal{O}(\sqrt{d \log m/m})$ with high probability.

Cuckoo hashing. Cuckoo hashing Pagh and Rodler (2001) can be viewed as a method to mitigate hash collisions. It involves using multiple hash functions, denoted as $H_1, \ldots, H_w$, to distribute n items into a hash table CH. Each bin in CH can hold at most one item. When inserting an item e, cuckoo hashing randomly selects a hash function H_i and maps e to the bin indexed by $H_i(e)$. If the target bin is occupied by e', e' is evicted and replaced with e. Then, e' is rehashed using a different hash function until an empty bin is found. If insertion fails after several attempts, the evicted item is stored in a buffer pool known as the stash. The works of Pinkas et al. (2018b); Kerschbaum et al. (2023) demonstrate that when employing $w = 3$ hash functions and setting the hash table size to $m = 1.27n$, the stash size can be effectively reduced to zero.

2.6 Security Model

In this paper, we consider security against malicious adversaries under a relaxed definition, which is the same as previous FHE-based PSI (Chen et al. 2018; Cong et al. 2021). Specifically, our protocol, JAGUAR, guarantees security against malicious clients and privacy against a malicious server. The goal of our security model is to prove that the malicious adversary has no impact on the computational result or the privacy of the sensitive inputs. Formally, we model and prove the security of our protocols under the universal composition (UC) framework.

3 Divide-and-Combine (DaC)

One of the major challenges in FHE-based PSI is reducing the computation cost without increasing the communication. Building on the partitioning optimization (Chen et al. 2017), we introduce the DaC method to lower the computation cost without incurring additional communication.

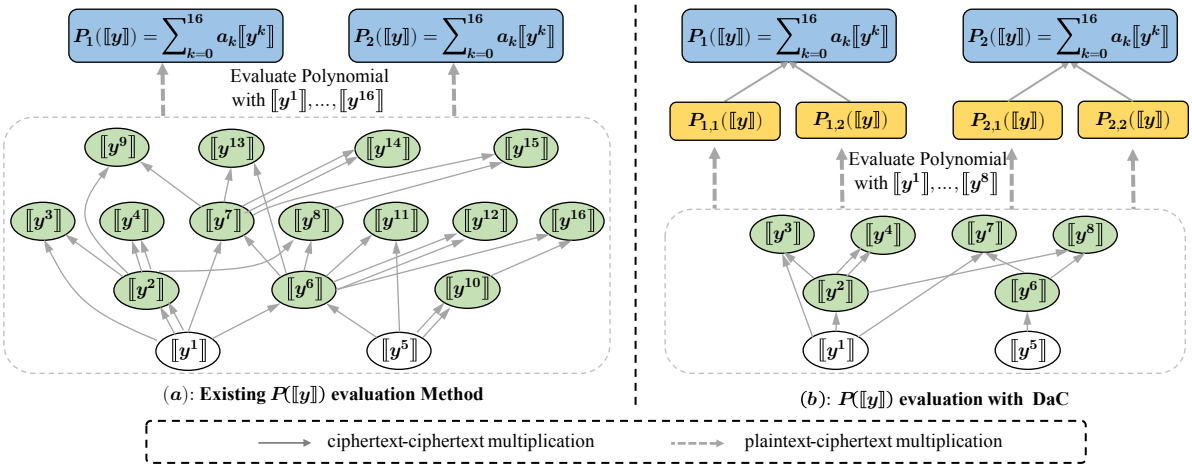

Fig. 4 Two different ways for the server to evaluate polynomial $P([\![y]\!])$ homomorphically in the case $B = 32, \alpha = 2$ and $\beta = 2$.

After applying the partitioning method, each bin of the server has been split into α subsets, denoted as $B_S^i[\cdot], i \in [\alpha]$. In DaC, we further divide each subset $B_S^i[\cdot], i \in [\alpha]$ into β smaller parts $B_S^{i,j}[\cdot], j \in [\beta]$, as specified by Equation (1), and build a polynomial $P_{i,j}(\cdot)$ for each of $B_S^{i,j}[\cdot]$.

$$B_S^i[\cdot] = \bigcup_{j\in[\beta]} B_S^{i,j}[\cdot] \quad i \in [\alpha], j \in [\beta] \tag{1}$$

After the division phase, both the polynomial degree and the number of holomorphic ciphertext multiplications for power calculation are reduced from $\mathcal{O}(B/\alpha)$ to $\mathcal{O}(B/(\alpha \cdot \beta))$, resulting in improved computational efficiency. However, the number of $P([\![y]\!])$ increases from α to $\alpha \cdot \beta$, which results in additional communication costs according to Chen et al. (2017). To eliminate the communication increase, DaC introduces an additional homomorphic multiplication for $P_{i,j}([\![y]\!])$ in each subset $B_S^i[\cdot], i \in [\alpha]$ to combine the β results into one. As shown in Equation (2), β polynomial ciphertext results $P_{i,j}([\![y]\!]), j \in [\beta]$ are combined into $P_i([\![y]\!])$ during the second step of DaC.

$$P_i([\![y]\!]) = \prod_{j\in[\beta]} P_{i,j}([\![y]\!]), \quad i \in [\alpha] \tag{2}$$

To illustrate the effectiveness of our method, consider a concrete example with bin size $B = 32$ and $\alpha = 2$. In this case, each bin is divided into α sub-bins, and the server evaluates the polynomial $P_i([\![y]\!]) = \sum_{k=0}^{B/\alpha} a_k \cdot [\![y^k]\!], i \in [\alpha]$ for each sub-bin. This requires ciphertext of powers $[\![y^1]\!]$ through $[\![y^{16}]\!]$ for each client item y as $B/\alpha = 16$. Suppose the client provides the encrypted powers $[\![y^1]\!]$ and $[\![y^5]\!]$ as part of the query. As shown in Figure 4(a), the baseline approach performs 14 ciphertext-ciphertext homomorphic multiplications to compute the necessary powers and produces two ciphertext outputs, $P_1([\![y]\!])$ and $P_2([\![y]\!])$, with a multiplication depth of 3.

In contrast, applying our DaC method with $\beta = 2$ further divides each sub-bin into two tiny bins. This reduces the maximum required exponent to $B/(\alpha\beta) = 8$, meaning the server only needs to compute up to $[\![y^8]\!]$. As depicted in Figure 4(b), DaC uses 6 ciphertext-ciphertext multiplications to compute the necessary powers and an additional 2 multiplications to aggregate every β evaluations into one ciphertext for

each sub-bin. Overall, DaC reduces the number of ciphertext-ciphertext homomorphic multiplications by approximately 42.8% compared to the prior method, while maintaining the multiplication depth and the number of final polynomial evaluation results. This demonstrates that DaC effectively reduces online computation overhead without introducing any additional communication.

More generally, we would like to find the optimal parameter β so that, when applying DaC, the server can execute the online computation process as efficiently as possible without exceeding the target depth.

Previous work (Chen et al. 2017) splits a hash bin into α subsets for fewer homomorphic multiplication operations. Each subset contains $\lceil B/\alpha \rceil$ items, which results in $\lceil B/\alpha \rceil - s$ ciphertext-ciphertext homomorphic multiplications in each bin, where s denotes the number of ciphertexts sent by the client. In contrast, when DaC is applied and each subset is further divided into β parts, only $\lceil B/(\alpha \cdot \beta) \rceil - s$ homomorphic multiplications are required. After the polynomial evaluation, DaC additionally combines every group of β ciphertexts into one, requiring $\alpha(\beta - 1)$ homomorphic multiplications, to avoid increasing communication overhead. Compared to CMDG+21, the multiplication reduction of our method in the online phase is

$$Reduce = \lceil \frac{B}{\alpha} \rceil + \alpha - (\lceil \frac{B}{(\alpha\beta)} \rceil + \alpha\beta) \quad (3)$$

According to the inequality of arithmetic and geometric means, we have:

$$(\lceil \frac{B}{(\alpha\beta)} \rceil + \alpha\beta) \geq 2\sqrt{\frac{B}{(\alpha\beta)} \cdot \alpha\beta} = 2\sqrt{B} \quad (4)$$

The equality holds if and only if $B/(\alpha\beta) = \alpha\beta$. Based on Equation (3) and Equation(4), we conclude:

$$Reduce \leq \lceil \frac{B}{\alpha} \rceil + \alpha - 2\sqrt{B} \quad (5)$$

The equality holds if and only if $\beta = \sqrt{B}/\alpha$. We choose the integer $\beta = \lfloor \sqrt{B}/\alpha \rfloor$ to improve the efficiency.

Since larger parameters, which increase both the communication and computation complexity, are required for deeper multiplication levels, we should make sure that the multiplication depth of our protocol is not exceed that of CMDG+21. For a selected β value, this constraint is expressed as:

$$Depth(\lceil \frac{B}{\alpha\beta} \rceil) + \lceil \log_2 \beta \rceil \leq Depth(\lceil \frac{B}{\alpha} \rceil) \quad (6)$$

where $Depth(\cdot)$ indicates the multiplication circuit depth of different degree polynomials. Since the postage stamp problem (Challis and Robinson 2010) , which determines the minimal-depth multiplication strategy, is generally hard to solve, the values of Depth$(\cdot)$ must be obtained from the table provided in Appendix A of Challis and Robinson (2010). Once the depth values are known, we verify whether Equation (6) holds. If it does not, β must be reduced (e.g., halved iteratively) until $\beta = 1$. The formal procedure for selecting β is described in Algorithm 1.

Algorithm 1 Optimal β selection

Input: Each bin maxload B, Partitioning parameter α
Output: The value of β

$\beta = \lfloor \sqrt{B}/\alpha \rfloor$
while $\beta > 1$ **do**
  Gets $Depth(\lceil \frac{B}{\alpha\beta} \rceil)$ and $Depth(\lceil \frac{B}{\alpha} \rceil)$ by looking up the table.
  $d = \lceil \log_2 \beta \rceil$
  if $Depth(\lceil \frac{B}{\alpha\beta} \rceil) + d \leq Depth(\lceil \frac{B}{\alpha} \rceil)$ **then**
    return β
  end if
  $\beta = \lfloor \beta/2 \rfloor$
end while

3.1 Complexity Analysis

With setting $\beta = \sqrt{B}/\alpha$ of the DaC method, the total homomorphic ciphertext multiplication for each bin is equal to

$$\begin{aligned} \lceil \frac{B}{\alpha\beta} \rceil + \alpha(\beta - 1) &= \lceil \frac{B}{\alpha\frac{\sqrt{B}}{\alpha}} \rceil + \alpha\frac{\sqrt{B}}{\alpha} - \alpha \\ &= \lceil \sqrt{B} \rceil + \sqrt{B} - \alpha \\ &< 2\sqrt{B} + 1 - \alpha \end{aligned} \quad (7)$$

In total, the complexity of homomorphic ciphertext multiplication is $\mathcal{O}(\sqrt{|X|})$ for the whole PSI protocol.

Note that CMDG+21 also achieves a ciphertext multiplication complexity of $\mathcal{O}(\sqrt{|X|})$ by employing the Paterson-Stockmeyer (PS) algorithm (Paterson and Stockmeyer 1973) for polynomial evaluation. However, our DaC method achieves better practical efficiency. We explain the reason for this improvement in detail below.

For simplicity, we focus on a single hash bin. Specifically, the scheme of CMDG+21 incurs a concrete homomorphic ciphertext multiplicative cost of $2\sqrt{B + B/\alpha + 2\alpha + 2}$ (see Appendix A of CMDG+21). In contrast, according to Equation (7), our DaC method requires only $2\sqrt{B} + 1 - \alpha$ homomorphic ciphertext multiplications. This reduction in multiplicative cost results in improved efficiency in practice.

4 Fixed VOLE-based OPRF

The work of Chen et al. (2018) employs an OPRF preprocessing phase, which maps private input sets X, Y to pseudorandom sets X', Y', to achieve improved performance and malicious security. However, as discussed in Section 1, the existing OPRF approaches are non-reusable across multiple clients (Chase and Miao 2020; Raghuraman and Rindal 2022). In this section, we propose the FV-OPRF, a lightweight and reusable OPRF protocol, to perform the preprocessing phase. As a building block, we first present the malicious fixed VOLE protocol in section 4.1. Then, we propose the full FV-OPRF in section 4.2.

4.1 Malicious fixed VOLE

The fixed VOLE protocol, described in section 2.3, achieves semi-honest security and can be easily extended to the unbalanced case of the client-server settings. Here, we present an extension of this protocol to defend against malicious clients. For simplicity, we assume that the server, which holds $\boldsymbol{c}, \boldsymbol{a}$, interacts with only one client, who holds $\Delta, \boldsymbol{b}$. The multi-client setting can be handled by executing this two-party protocol multiple times.

Our key point is employing consistency check techniques to detect potentially malicious behavior in VOLE generation. Concretely, we want to check that $\boldsymbol{c}[i] = \Delta \cdot \boldsymbol{a}[i] + \boldsymbol{b}[i]$. The parties achieve this by sampling a vector $\boldsymbol{\chi} = \{\chi_1, ..., \chi_d\} \in \mathbb{F}^d$ and checking that

$$\chi_i \cdot \boldsymbol{c}[i] = \chi_i \cdot \Delta \cdot \boldsymbol{a}[i] + \chi_i \cdot \boldsymbol{b}[i],\ i \in [d]$$

This check can be done by leveraging the FHE scheme. The server uses FHE to encrypt the random number χ_i and sends the ciphertexts $[\![\chi_i]\!]$, $a^*[i] = \chi_i \cdot \boldsymbol{a}[i]$, $i \in [d]$ to the client, who computes $[\![b^*[i]]\!] = [\![\chi_i]\!] \cdot \boldsymbol{b}[i] + \Delta \cdot a^*[i]$ and returns the results. Then, the server decrypts the ciphertext $[\![b^*[i]]\!]$ to $b^*[i]$ and defines $c^*[i] = \chi_i \cdot \boldsymbol{c}[i]$. Finally, the server checks whether $b^*[i] = c^*[i]$ and abort if the check fails. For a VOLE tuple with the length of d, the communication of the consistency check is $2d$ ciphertexts and d plaintexts.

To optimize the communication complexity, we propose a batched consistency check. For the verification of the VOLE with length d, we instruct two parties to compress the vectors $\boldsymbol{a}^*, [\![\mathbf{b}^*[i]]\!]$, $i \in [d]$, into one number, respectively. Concretely, the server first computes $a^* = \sum_{i=1}^{d} \chi_i \cdot \boldsymbol{a}[i]$ and sends $([\![\chi_1]\!], ..., [\![\chi_d]\!]), a^*$ to the client. Consequently, the client computes and returns $[\![b^*]\!] = \sum_{i=1}^{d} [\![\chi_i]\!] \cdot \mathbf{b}[i] + \Delta \cdot a^*$ back. Finally, the server decrypts $[\![b^*]\!]$ and checks whether $b^* = \sum_{i=1}^{d} \chi_i \cdot \mathbf{c}[i]$. In this way, the communication cost of the batch consistency check for a VOLE with size d is reduced to $d + 1$ ciphertexts and 1 plaintext.

The details of the malicious fixed VOLE are shown in Figure 5 and the proof is given in Appendix B.1.

Theorem 1 *If the FHE scheme is IND-CPA secure, the protocol* $\Pi_{\text{MalFixedVOLE}}$ *of Figure 5 is secure against a malicious client and provides privacy against a malicious server in the random oracle,* $\mathcal{F}_{\text{FixedVOLE}}$*-hybrid model.*

Proof Sketch. Our protocol achieves security against a malicious client while preserving privacy against a malicious server. To detect client misbehavior, the protocol incorporates a batched consistency check. In the simulation-based proof, a simulator *Sim* emulates the ideal functionality $\mathcal{F}_{\text{MalFixedVOLE}}$ and verifies whether the response from an adversarial client aligns with the expected aggregated result.

Protocol $\Pi_{\text{MalFixedVOLE}}$

Public Params: The length of the vector d and the client number s

Parties: The server and s clients.

Protocol:

VOLE Generation:

1) Two parties invoke the $\mathcal{F}_{\text{FixedVOLE}}$ with vector length d and the VOLE instance number s. The server plays as P_0 and the client plays as P_1. As a result, the server gets $\boldsymbol{c}_i, \boldsymbol{a} \in \mathbb{F}^d$ and the i-th client gets $\Delta_i \in \mathbb{F}, \boldsymbol{b}_i \in \mathbb{F}^d$ for $i \in [s]$.

Consistency Check:

2) The server samples $\{\chi_1, ..., \chi_d\} \in_R \mathbb{F}^d$ and computes $a^* = \sum_{j=1}^{d} \chi_j \cdot \boldsymbol{a}[j]$. Then, it uses the FHE to encrypt $\{\chi_1, ..., \chi_d\}$, and sends $[\![\chi_1]\!], ..., [\![\chi_d]\!]$, a^* to all clients.
3) For each $i \in [s]$, the i-th client computes $[\![\mathbf{b}_i^*]\!] = \sum_{j=1}^{d} [\![\chi_j]\!] \cdot \boldsymbol{b}_i[j] + \Delta_i \cdot a^*$ and returns $[\![b_i^*]\!]$ back to the server.
4) The server decrypts the ciphertext $[\![b_i^*]\!]_{i\in[s]}$ to $\{b_i^*\}_{i\in[s]}$ and defines $c_i^* = (\sum_{j=1}^{d} \chi_j \cdot \boldsymbol{c}_i[j])$, $i \in [s]$. Finally, the server check whether $b_i^* = c_i^*$. If it is *false*, then abort.

Fig. 5 Protocol $\Pi_{\text{MalFixedVOLE}}$ of Malicious Fixed VOLE

Specifically, the simulator Sim first interacts with the ideal functionality $\mathcal{F}_{\text{FixedVOLE}}$ to obtain $\boldsymbol{b}i$ and Δ_i from the i-th client. In Step 2, Sim samples random values $\{\chi'_j\}_{j\in[s]}$ and a' from the field $\mathbb{F}$ to simulate the values $\{\chi_j\}_{j\in[s]}$ and a^* in the real protocol. Due to the IND-CPA security of the underlying FHE scheme, these simulated ciphertexts are computationally indistinguishable from the real ones.

In this simulation, the only way a malicious client can pass the consistency check without honestly following the protocol is by correctly guessing both challenge values: $v_1 = \sum_{j=1}^{d} \chi'_j \cdot \boldsymbol{b}_i[j]$ and $v_2 = a'$. However, since both χ'_j and a' are sampled uniformly at random and are hidden from the adversary, the probability of successfully guessing both values is at most $1/|\mathbb{F}|$, which is negligible for a large field $\mathbb{F}$ (e.g., 2^{64}).

In many applications—including our protocol—security at the higher level can only be compromised if the adversary learns the vector $\boldsymbol{a}$ or $\boldsymbol{c}$. As such, eliminating selective-failure leakage on individual elements of these vectors is not strictly necessary. Consider an adversary attempting a selective-failure attack to infer the value of $\boldsymbol{a}[i]$. Suppose the adversary guesses that $\boldsymbol{a}[i]$ lies in a subset $S \subset \mathbb{F}$, and the protocol aborts if the guess is incorrect (i.e., $\boldsymbol{a}[i] \notin S$). The probability that the guess is accepted—i.e., the attack does not trigger an abort—is $|S|/|\mathbb{F}|$. Given this event, the adversary's uncertainty about $\boldsymbol{a}[i]$ drops to $\log|S|$ bits. Thus, the probability of correctly identifying $\boldsymbol{a}[i]$ remains $1/|\mathbb{F}|$, which is identical to the success probability without any leakage.

4.2 Fixed VOLE-based OPRF

In the unbalanced case of the client-server setting, a reusable encoding of the server set X provides many practical benefits. For instance, such encoding can be distributed publicly using content distribution networks or peer-to-peer networks for improved accessibility.

To enable a reusable OPRF procedure, we integrate the fixed VOLE protocol into our VOLE-based OPRF construction. Concretely, the server and the multiple clients first invoke the fixed VOLE protocol $\Pi_{\text{MalFixedVOLE}}$ to generate VOLE tuples $\boldsymbol{c}_i = \Delta_i \cdot \boldsymbol{a} + \boldsymbol{b}_i$, $i \in [s]$, where the server holds $\boldsymbol{a}, \boldsymbol{c}_i$ for all $i \in [s]$ and the i-th client only holds $\Delta_i, \boldsymbol{b}_i$, with the same vector $\boldsymbol{a}$. All tuples share the same vector $\boldsymbol{a}$ on the server side. Since $\boldsymbol{p}$ is only related to the server set X, which remains the same for different clients, the encoding result $\boldsymbol{a'} = \boldsymbol{a} + \boldsymbol{p}$ of the server is the same for different clients. Therefore, the server could encode its set X into $\boldsymbol{a'}$ only once, and multiple clients could reuse the $\boldsymbol{a'}$ to compute the OPRF results of the items in their sets. The full protocol of FV-OPRF is shown in Figure 6.

Moreover, FV-OPRF requires only a constant number of public-key cryptographic operations and a linear number of symmetric-key operations. In addition, the reusability of the encoding result $\boldsymbol{a'}$ ensures that the server consistently operates on the same input set across multiple executions.

Protocol $\Pi_{\text{FV-OPRF}}$

Public Params: The length of the vector d and a set X with size $|X|$.

Parties: The server and s clients.

Protocol:

1) For each $i \in [s]$, i-th client samples $w_i^c \leftarrow \mathbb{F}$ and sends $\delta_i^c = H^{\mathbb{F}}(w_i^c)$ to the server.
2) The server samples $w^s \leftarrow \mathbb{F}$ and solve the systems

$$M \cdot \mathbf{p} = [H^{\mathbb{F}}(x_1), \ldots, H^{\mathbb{F}}(x_{|X|})]$$

where $M \in \mathbb{F}^{|X| \times d}$ and $M_i = row(x_i)$ s.t. $row(\cdot) : \mathbb{F} \to \mathbb{F}^d$.
3) Two parties invoke the $\mathcal{F}_{\text{MalFixedVOLE}}$ with vector length d and the VOLE instance number s. As a result, the server gets $\boldsymbol{c}_i, \boldsymbol{a} \in \mathbb{F}^d$ and the i-th client gets $\Delta_i \in \mathbb{F}, \boldsymbol{b}_i \in \mathbb{F}^d$ for $i \in [s]$.
4) The server publishes w^s, $\boldsymbol{a}' = \mathbf{p} + \boldsymbol{a}$.
5) For each $i \in [s]$, the i-th client gets the vector $\boldsymbol{a}'$ and defines $\boldsymbol{k} = \boldsymbol{b}_i + \boldsymbol{a}' \cdot \Delta_i$. Then, clients sends $w_i^c, i \in [s]$ to the server, who aborts if $\exists\ i \in [s]$ such that $\delta_i^c \neq H^{\mathbb{F}}(w_i^c)$. Finally, both parties define $w_i = w^s + w_i^c$.
6) For each $i \in [s]$, the i-th client outputs OPRF result of each item y as $F(y_j) = H(\langle row(y_j), \boldsymbol{k}_i \rangle + w_i - \Delta_i \cdot H^{\mathbb{F}}(y_j))$.
7) For each $i \in [s]$, the server outputs $X' = \{H(\langle row(x), \boldsymbol{c}_i \rangle + w_i) | x \in X\}$

Fig. 6 Protocol $\Pi_{\text{FV-OPRF}}$ of FV-OPRF

These properties make FV-OPRF a lightweight and reusable OPRF construction, particularly well-suited for an unbalanced client-server setting.

Theorem 2 *The protocol $\Pi_{\text{FV-OPRF}}$ of Figure 6 securely realizes the $\mathcal{F}_{\text{OPRF}}$ functionality against malicious clients and provides privacy against a malicious server in the presence of an adversary corrupting up to s-1 clients and a server in the random oracle, $\mathcal{F}_{\text{MalFixedVOLE}}$-hybrid model.*

Proof Sketch. As mentioned above, our protocol is obtained using a simple modification of the scheme of Rindal and Schoppmann (2021), i.e., using malicious fixed VOLE instead of general VOLE. Since there is no additional information revealed, their proof (Rindal and Schoppmann 2021, Section 3.2) can be trivially adapted to our protocol. We will give an overview here, but refer the reader to Rindal and Schoppmann (2021) for the full details.

For the malicious clients, at step 3, the *Sim* plays the role of $\mathcal{F}_{\text{MalFixedVOLE}}$ and gets the $\boldsymbol{b}_i, \Delta_i$ from the adversary $\mathcal{A}$. Then, on behalf of the server, *sim* sends uniform $\boldsymbol{a}'$ to $\mathcal{A}$. Since the vector $\boldsymbol{a}$ is uniformly distributed in the view of the $\mathcal{A}$ and the $\Pi_{\text{MalFixedVOLE}}$ is secure against malicious clients, this hybrid is indistinguishable from the simulation

For the malicious server, at step 3, the *Sim* plays the role of $\mathcal{F}_{\text{MalFixedVOLE}}$ and gets the $\boldsymbol{c}_i, \boldsymbol{a}$ from the adversary $\mathcal{A}$. Then, $\mathcal{A}$ sends a' to the clients in setp 4. At this point, *Sim* uses the random oracle to extract the server's input X. Concretely, *Sim* first computes $p = a' - a$. Then, for each of the random oracle queries made by $\mathcal{A}$, *Sim* checks if $\langle row(x), p \rangle = H^{\mathbb{F}}(x)$ and if so adds x to set X. After that, *Sim* sends X to the ideal functionality of OPRF and receives $\{F(x) | x \in X\}$. Finally, *Sim* samples random values to simulate the $w_i, i \in [s]$ in step 5. With the protocol $\Pi_{\text{MalFixedVOLE}}$ provides privacy against a malicious server, this hybrid is indistinguishable from the simulation.

5 Full Protocol of JAGUAR

The full description of JAGUAR is given in Figure 7. JAGUAR contains base, setup, and online phases. The base and setup phases are considered to be offline.

In the base phase, both parties agree on protocol parameters, including hash functions $h_1, \cdots, h_w$, the hash table length m, and the partitioning parameter α, and perform the fixed VOLE protocol $\Pi_{\text{MalFixedVOLE}}$.

In the setup phase, two parties perform the FV-OPRF evaluation described in Section 4. The server then successively uses simple hashing to insert the OPRF results of X into the hash table and splits each bin by partitioning. The server also needs to perform the dividing part of DaC

Protocol Π_{JAGUAR}

Input: The client R_k has an input set Y . The server S has an input set X. $|Y| \ll |X|$. The FHE parameters (n, q, t), hash functions $H_0, ..., H_w : [2^\sigma] \mapsto [2^{\sigma_1}]$, and a random oracle $F : \mathbb{F} \mapsto [2^\sigma]$.
Protocol:

I. **Base Phase:**

1. S and R_k agree on table length m, and the partitioning parameter α. Moreover, they invoke the $\mathcal{F}_{\text{MalFixedVOLE}}$, and S gets $\boldsymbol{c}_k, \boldsymbol{a} \in \mathbb{F}^d$ and R_k gets $\Delta_k \in \mathbb{F}, \boldsymbol{b}_k \in \mathbb{F}^d$.

II. **Offline Phase:**

2. [**OPRF**]: S performs the $\Pi_{\text{FV-OPRF}}$, maps X to a pseudorandom set X', and publishes the vector $\boldsymbol{p} + \boldsymbol{a}$.
3. [**Hashing**]: S performs simple hashing with hash functions $H_i, i \in [w]$ to map item of X' into the table SH, where each bin of the table contains B item.
4. [**Coefficient Computation**]: For i-th hash bin SH_i $(i \in [m])$, S uses the partitioning and DaC to split it into $\alpha\beta$ tiny-bins of size $B/(\alpha\beta)$, denoted $SH_{i,1}, ..., SH_{i,\alpha\beta}$. For each tiny-bin $SH[i, j]$, S constructs the polynomial $P_{i,j} = a_{i,j,0} \cdot x^{B/(\alpha\beta)} + \cdots + a_{i,j,B/(\alpha\beta)}$ such that $P_{i,j}(x) = 0$ for all $x \in SH_{i,j}$.

III. **Online Phase:**

5. [**OPRF**] R_k gets the $\boldsymbol{p} + \boldsymbol{a}$ and maps set Y to Y' by the $\Pi_{\text{FV-OPRF}}$.
6. [**Cuckoo hashing**] After OPRF, R_k uses hash functions $H_0, ..., H_w$ to insert the set Y' into the table CH by cuckoo hashing.
7. [**PSI Query**] R uses the public key pk to encrypt each item stored in the hash table CH and sends all ciphertexts $[\![y_i]\!]$ $(i \in [m])$ to S.
8. [**Polynomial Evaluation**] S receives the ciphertexts and computes all powers of the encryption $[\![y_i]\!], ..., [\![y_i^{B/\alpha}]\!]$ for each $[\![y_i]\!]$. For each tiny-bin $SH_{i,j}$, S samples a non-zero random number $r^*_{i,j}$ and homomorphically evaluates $P_{i,j}$ by $[\![z_{i,j}]\!] = r^*_{i,j} \cdot \sum_{t=0}^{B/(\alpha\beta)} \left(a_{i,j,t} \cdot [\![y_i^{(B/(\alpha\beta)-t)}]\!] \right)$.
9. [**DaC-Combine**] S combines every β polynomial evluation results $[\![z_{i,j}]\!]$, $i \in [m], j \in [\alpha\beta]$ into one and gets $[\![z'_{i,j}]\!]$, $i \in [m], j \in [\alpha]$. All evaluation results are sent back to R
10. [**Intersection Output**] R decrypts all ciphertexts $[\![z'_{i,j}]\!]$ to $z'_{i,j}$ $(i \in [m], j \in [\alpha])$. For each $y' \in Y'$, stored in i-th bin, if $\exists\ j \in [\alpha]$ such that $z'_{i,j} = 0$, R outputs the original item y of y' as the intersection item.

Fig. 7 Full protocol Π_{JAGUAR} of JAGUAR (based on Chen et al. (2017, 2018); Cong et al. (2021))

and construct the polynomial $P(y)$. Meanwhile, the client executes the cuckoo hash for each OPRF result of Y and creates the query $[\![y]\!]$ by FHE.

The reusable encoding of X makes JAGUAR highly flexible because the client can access the encoding anytime. Furthermore, it ensures that a consistent set X is used across all clients.

The online phase consists of polynomial evaluation and decryption. After receiving the query, the server will evaluate the polynomial $P([\![y]\!])$. According to DaC, every β of them will be combined into one ciphertext. Then, the results will be sent back to the client, who decrypts the results and checks if $P(y)$ equals zero to obtain the intersection result. Importantly, the optimizations of previous works (Chen et al. 2017, 2018) and CMDG+21 can be used to improve the protocol efficiency.

The security model of JAGUAR follows the work of CMDG+21, where the PSI protocol is

secure against malicious clients and provides privacy against a malicious server in the dishonest-majority setting (i.e., the adversary can corrupt up to s-1 parties of the s parties). Theorem 3 captures the security of JAGUAR, and the ideal functionalities of PSI are presented in Figure 8

Theorem 3 *Assuming s clients and a server in the protocol. The protocol Π_{JAGUAR} of Figure 7 securely realizes the $\mathcal{F}_{\text{PSI}}$ functionality against malicious clients and provides privacy against a malicious server in the presence of an adversary corrupting up to s-1 clients and a server in the random oracle, $\mathcal{F}_{\text{FV-OPRF}}$-hybrid model.*

Proof Sketch. We now sketch the security proof of the JAGUAR protocol under the UC framework, and the full proof of Theorem 3 is given in Appendix B.2. JAGUAR follows the same high-level structure as prior maliciously secure uPSI protocols such as Chen et al. (2018) and CMDG+21, which adopt a two-phase design: (1) an offline oblivious pseudorandom function (OPRF) evaluation, and (2) an online homomorphic polynomial evaluation phase. Therefore, the overall security of JAGUAR can be established by proving that our modifications, FV-OPRF and DaC, preserve the security properties of the original design. Since DaC is purely a local optimization on the server side and introduces no additional communication, it has no effect on the security guarantees. As such, the main focus of our security analysis lies in the security of the FV-OPRF construction, which has been formally proven in Theorem 3. Therefore, we conclude that JAGUAR securely realizes the PSI functionality against malicious clients and provides privacy against a malicious server.

Functionality $\mathcal{F}_{\text{PSI}}$

Input: $\mathcal{F}_{\text{PSI}}$ receives set X, Y from the client and server, respectively.
Functionality: $\mathcal{F}_{\text{PSI}}$ computes Compute $X \cap Y$. Then it outputs the result to the client and nothing to the server.

Fig. 8 Ideal functionality $\mathcal{F}_{\text{PSI}}$ of private set intersection

5.1 Variants and Extensions

The depicted protocol is a general version that uses both DaC and FV-OPRF. We define two variants of JAGUAR:

1. **DaC**. In this variant, we only use the DaC method to optimize the online computation cost of the server. Following CMDG+21, we employ DH-based OPRF, which ensures that the protocol does not require offline communication. However, similar to CMDG+21, this variant necessitates an additional round of communication during the online phase to compute the OPRF results of the client set.
2. **DaC+FV**. In this variant, we retain both the DaC optimization and the FV-OPRF construction. Compared to the DaC-only variant, this version introduces a one-way communication step in the offline phase. Crucially, it supports a lightweight and reusable OPRF phase, along with an efficient preprocessing phase for OPRF computation. These features make this variant better suited for unbalanced client-server scenarios, where the server handles a large dataset and serves multiple lightweight clients with small sets.

Both protocol versions, each with its own strengths, highlight the flexibility of JAGUAR in adapting to different scenario requirements.

6 Experiment

We implemented JAGUAR to demonstrate the feasibility of the proposed techniques. For the *leveled*-FHE, we adopt the SEAL library SEAL (2022), which provides a C++ implementation of the BFV scheme Fan and Vercauteren (2012). The parameters for *leveled*-FHE scheme and corresponding security levels k follow those in CMDG+21. The encryption parameters used in this section support at least 128 bits of security.

We begin by evaluating our protocol under various parameter settings to demonstrate its overall performance. We then compare JAGUAR with the current SOTA uPSI protocols, including KRS+19, CMDG+21, and WY23 (Wu and Yuen 2023), highlighting its computational advantages.

Finally, we extend our evaluation to the multi-client setting to further demonstrate the scalability and practicality of our approach. Following the evaluation settings of KRS+19, CMDG+21, and WY23, we consider unbalanced cases where the client set size $|Y| \in \{5535, 11041\}$ and the server set size $|X| \in \{2^{20}, 2^{24}\}$. Each element is represented using 80 bits.

Our experiments are executed on an Intel(R) Xeon(R) Silver 4314 CPU with 2.40GHz and 128GB RAM machine server. Specifically, we consider a LAN setting with 10 Gbps throughput and a 0.2ms round-trip time (RTT). We also consider three WAN settings with 100 Mbps, 10 Mbps, and 1 Mbps bandwidth, each with an 80 ms RTT. Network bandwidth and latency are simulated using the Linux Traffic Control (TC) tools. For a fair comparison with related work, all experiments—both offline and online phases—are executed on a single thread.

In all experiments, we report communication overhead in megabytes (MB), calculated based on the number and size of all messages transmitted during both the offline and online phases. For the runtime, we report in seconds.

6.1 Efficiency Results

We first benchmark the DaC variant protocol of JAGUAR with various β to show the improvements of our DaC optimization. The detailed results are shown in Table 1. To facilitate a detailed and reproducible evaluation, we report the time cost of each major component in the online phase, including client-side encryption (Enc.), decryption (Dec.), and communication in both directions ($C \to S$ and $S \to C$). For the server, we separate the latency into offline and online computation. Additionally, to reflect runtime variability and ensure reproducibility, we report the standard deviation for the server's runtimes. These metrics enable us to identify both computational and communication bottlenecks and to determine which parts of the protocol contribute most significantly to overall performance.

To empirically validate the optimal choice of $\beta = \lfloor \sqrt{B/\alpha} \rfloor$ as motivated in Equation (7), we conduct an ablation study on various β. When $\beta = 1$, the computation essentially falls back to the baseline construction used in CMDG+21. This serves as a reference point for evaluating the benefit of our approach. Notably, DaC consistently achieves the best performance when $\beta = 2$, meaning that $\beta = 2$ captures most of the benefit from DaC without increasing the multiplicative depth in our set size setting.

Specifically, in our experimental setting, the ratio $\sqrt{B}/\alpha$ is typically no more than 8. However, the setting of $\beta = 4$ or 8 leads to a deeper homomorphic multiplication circuit, which significantly degrades performance. This is evident from Table 1. When $\beta = 4$, all cases experience an increase in multiplicative depth (from 3 or 4 to 4 or 5), along with substantial slowdowns in both online computation and communication cost.

Moreover, larger multiplicative depths require higher noise budgets in the underlying FHE scheme to support correct decryption, which in turn increases ciphertext sizes. This explains the corresponding growth in communication costs.

6.2 Comparison with SOTA uPSI Protocols

In this part, we compare JAGUAR with the SOTA uPSI protocols in a classical 2PC setting.

Compare to CMDG+21: As shown in Table 2, the DaC variant of our JAGUAR protocol achieves a 1.6× speedup in the online phase compared to the work of CMDG+21 in the LAN setting. This improvement primarily stems from our DaC method, which effectively reduces the number of homomorphic multiplications during the online computation. Similar to CMDG+21, the DaC variant adopts a DH-based OPRF in the offline phase, which incurs no communication overhead during that phase. As a result, the offline cost of our DaC variant remains comparable to that of CMDG+21.

In contrast, our DaC+FV variant shows significant improvements in both the offline and online phases. Specifically, its offline phase is 4.9× faster than CMDG+21, owing to the efficiency of our lightweight FV-OPRF. The FV-OPRF requires only a constant number of public-key cryptographic operations and a linear number of symmetric-key operations, making it substantially more efficient than traditional DH-based constructions. Moreover, unlike CMDG+21, our protocol executes the entire OPRF process during

Table 1 Benchmarks of our DaC variant protocol. All experiments are implemented with single threads. $C \to S$ and $S \to C$ denote the communications from the client to the server, and from the server to the client. Parenthetical values indicate runtime variability (measured across repeated runs).

X	Y	β	Mult. Depth	Server Runtime (s)		Client		Comm. (MB)	
				Offline Processing	Online Evaluation	Enc.	Dec.	$C \to S$	$S \to C$
2^{20}	5535	1	3	32.5 (±2.1)	5.3 (±0.2)	0.1	0.2	3.6	1.8
		2	3	32.4 (±2.7)	3.1 (±0.1)	0.1	0.2	3.6	1.8
		4	4	30.9 (±2.3)	5.8 (±0.2)	0.1	0.3	3.7	2.3
	11041	1	3	32.6 (±2.2)	9.9 (±0.4)	0.4	0.3	6.6	2.3
		2	3	32.7 (±3.1)	5.1 (±0.2)	0.4	0.3	6.6	2.3
		4	4	31.8 (±2.6)	11.9 (±0.5)	0.6	0.4	6.8	3.4
2^{24}	5535	1	4	564 (±23)	17.9 (±1.2)	1.0	0.7	6.3	2.2
		2	4	525 (±21)	14.4 (±1.6)	1.0	0.7	6.3	2.2
		4	5	503 (±17)	27.7 (±2.5)	1.6	0.9	10.4	4.2
	11041	1	4	564 (±26)	28.7 (±1.9)	1.8	0.8	11.7	2.9
		2	4	529 (±25)	16.3 (±1.7)	1.8	0.8	11.7	2.9
		4	5	511 (±20)	34.6 (±2.1)	2.5	1.1	20.5	5.7

the offline phase, enabled by a one-way communication model.

Additionally, the DaC+FV variant achieves up to 2.7× speedup (and 2× on average) in the online phase over CMDG+21 in the LAN setting. This improvement results not only from the DaC optimization but also from the FV-OPRF design, which eliminates the need for interactive OPRF computation for the client's set during the online phase. Furthermore, our protocol incurs slightly lower communication overhead compared to CMDG+21, contributing to better overall performance.

Compare to WY23: Recently, WY23 proposed the virtual bloom filter (VBF) and polynomial links (PoL) methods to reduce the overhead of FHE-based PSI protocols. Since no available code was provided, we directly used the data from their paper to evaluate performance. Among the three proposed protocol versions, we selected the VBF+slicing-based protocol for comparison, since the VBF+PoL and PoL-based protocols had excessive offline overhead. As shown in Table 2, while the protocol by WY23 has better online communication overhead when $|X| = 2^{24}$, both variants of the JAGUAR protocol consistently outperform it in terms of runtime for $|X| = \{2^{20}, 2^{24}\}$ across different network environments. Specifically, in LAN, JAGUAR is 1.2× faster than WY23 on average. Importantly, our optimization methods are compatible with the PoL and VBF. Therefore, combining these methods can potentially yield even more efficient protocols. We delay this as part of future work.

Compare to KRS+19: This work uses OPRF and CF to construct the PSI protocol. Similar to the DaC+FV variant of JAGUAR, the LowMC-GC-PSI and ECC-NR-PSI protocols involve offline communication. For the offline phase, our DaC+FV protocol achieves faster runtime, while their protocol offers a marginally lower communication overhead. For the online phase, the DaC+FV variant of JAGUAR takes less communication and outperforms the LowMC-GC-PSI and ECC-NR-PSI protocols of KRS+19 in WAN. Compared to them, JAGUAR (DaC+FV) achieves a speedup of 6.5× and 1.7× on average in the online phase over 100 Mbps while reducing the communication cost on average of 22.7× and 5.8×, respectively. This demonstrates that JAGUAR (DaC+FV) achieves a better balance between computation and communication. Moreover, the protocol of KRS+19 suffers from the problem of non-negligible FPR, which is 2^{-29} in the experiment shown in Table 2.

We also acknowledge that Resende and Aranha (2021) proposed an efficient uPSI protocol utilizing a rank-and-select-based quotient filter

Table 2 Comparison to the SOTA uPSI, KRS+19, CMDG+21, and WY23. All experiments are implemented with single threads. Online Comm. (MB) includes the client-to-server and server-to-client communication costs. The 'GC' and 'NR' refer to the LowMC-GC-PSI and ECC-NR-PSI protocols of KRS+19, respectively.

$\|X\|$	$\|Y\|$	**Protocol**	**Offline Times**	**Offline Comm.**	**Online Times**				**Online Comm.**
					10 Gbps	100 Mbps	10 Mbps	1 Mbps	
2^{20}	5535	KRS+19 (GC)	8.5	4.2	5.6	25.6	215.3	2136.6	129.7
		KRS+19 (NR)	262.0	4.2	5.9	12.1	87.2	866.8	32.7
		CMDG+21	32.5	**0**	5.6	5.9	6.9	23.1	5.4
		WY23	26.3	**0**	3.7	3.7	4.0	18.7	5.3
		JAGUAR (DaC)	32.4	**0**	3.4	3.6	4.2	20.8	5.4
		JAGUAR (DaC+FV)	**6.8**	5.2	**2.6**	**2.7**	**3.6**	**18.1**	**5.1**
	11041	KRS+19 (GC)	8.5	4.2	12.5	52.8	430.3	4359.1	258.8
		KRS+19 (NR)	262.0	4.2	11.9	19.7	119.2	860.3	65.2
		CMDG+21	32.6	**0**	10.6	11.1	13.8	26.4	8.9
		WY23	26.6	**0**	5.1	5.1	5.7	25.2	8.9
		JAGUAR (DaC)	32.7	**0**	5.8	6.3	8.2	21.6	8.9
		JAGUAR (DaC+FV)	**7.9**	5.2	**4.0**	**4.4**	**5.1**	**19.3**	**8.3**
2^{24}	5535	KRS+19 (GC)	117	67	**5.6**	25.9	215.4	2133.3	129.7
		KRS+19 (NR)	3298	67	5.9	**11.9**	87.3	867.8	32.7
		CMDG+21	564	**0**	19.6	20.8	21.1	28.4	8.5
		WY23	684	**0**	16.8	16.9	17.1	25.8	**6.9**
		JAGUAR (DaC)	525	**0**	16.1	16.6	16.9	25.1	8.5
		JAGUAR (DaC+FV)	**97**	83	15.8	16.2	**16.5**	**23.3**	8.2
	11041	KRS+19 (GC)	117	67	12.5	53.5	413.2	4249.0	258.8
		KRS+19 (NR)	3298	67	**11.9**	19.9	119.2	1153.3	65.2
		CMDG+21	564	**0**	31.3	32.1	32.6	43.2	14.6
		WY23	661	**0**	21.4	21.5	22.1	34.6	**11.8**
		JAGUAR (DaC)	529	**0**	18.9	19.3	19.7	31.1	14.6
		JAGUAR (DaC+FV)	**109**	83	18.1	**18.3**	**18.9**	**30.1**	14.0

(RSQF), achieving a highly efficient online phase. Since it only achieves semi-honest security, we did not include it in our comparison. Additionally, their protocol incurs significant offline costs to achieve a negligible FPR. When $|X| = 2^{24}$, 225 MB is required to achieve an FPR of 2^{-40}.

6.3 Scalability in unbalanced client-server Scenario

JAGUAR is suitable for the unbalanced client-server scenario, where the server with a large set repeatedly runs PSI with many clients with small sets. Therefore, we perform comparisons in this setting. This subsection employs a **1-to-N** model, where each client independently initiates set intersection queries simultaneously, while the server processes requests serially via a single thread. The server holds a fixed input set, while each client holds a distinct, relatively small set. leveraging the FV-OPRF protocol, the server encodes its dataset once into a cross-client consistent reusable structure (as discussed in Section 4.2), effectively amortizing encoding overhead while enabling clients to efficiently compute pseudorandom mappings. Since WY23 has no available implementation, in this section, we compare JAGUAR only to the protocols of KRS+19 and CMDG+21.

Compare to KRS+19. Since the protocols of KRS+19 involve offline communication, we compare them with our DaC+FV variant of JAGUAR in Figures 9(a)–9(c). In the LAN setting, JAGUAR outperforms both ECC-NR-PSI and LowMC-GC-PSI in terms of runtime, and its advantage becomes more pronounced as the

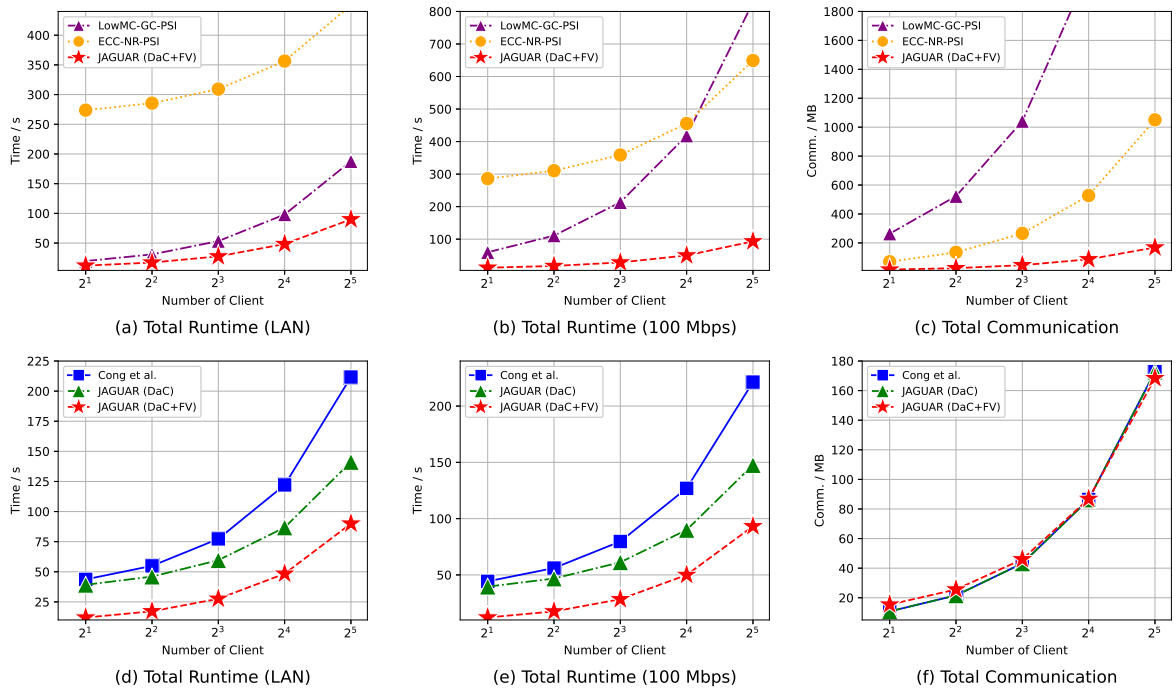

Fig. 9 Total cost (including online and offline) of JAGUAR and other uPSI (Kales et al. 2019; Cong et al. 2021) with server set $|X| = 2^{20}$ and client set $|Y| = 5535$ in the multi-client setting. Runtime is in seconds and Communication is in MB.

number of clients increases. Notably, JAGUAR achieves a substantial performance gain over ECC-NR-PSI, and runs approximately twice as fast as LowMC-GC-PSI. This indicates that JAGUAR can efficiently scale to support multiple clients without incurring significant additional processing time.

Moreover, as shown in Figure 9(c), JAGUAR incurs significantly lower communication overhead compared to both ECC-NR-PSI and LowMC-GC-PSI. This reduced bandwidth consumption makes JAGUAR especially suitable for scenarios with limited bandwidth. As a result, in the 100 Mbps setting, JAGUAR consistently maintains a lower total runtime than the other protocols, demonstrating both superior scalability and practicality.

Compare to CMDG+21. For the work of CMDG+21, we compare it not only with the DaC+FV variant but also with the DaC variant of our protocol, as shown in Figure 9(d)-9(f). Compared to CMDG+21, both variants of JAGUAR demonstrate superior efficiency in various network settings. When the number of clients is 32, the DaC+FV variant protocol (resp. DaC variant protocol) is 1.8× and 1.9× (resp. 1.6× and 1.5×) faster than CMDG+21 in LAN and 100 Mbps networks, respectively. This suggests that JAGUAR handles increasing client loads more efficiently without incurring substantial computation overhead. Notably, the communication overhead of the DaC+FV variant protocol is comparable to, but slightly lower than, that of CMDG+21. These results collectively demonstrate the scalability and practicality of JAGUAR in real-world uPSI applications.

7 Related work

The earliest PSI was proposed by Meadows (1986) and Huberman et al. (1999). These PSI protocols are based on public-key cryptography and utilize the Diffie-Hellman (DH) key exchange method (Diffie and Hellman 1976). However, the first formal PSI protocol was proposed by Freedman et al. (2004) in the standard model. This approach is based on the oblivious polynomial evaluation and leverages partially homomorphic encryption. Recently, many protocols (Pinkas et al. 2014; Kolesnikov et al. 2016; Pinkas et al. 2018b, 2020;

Rindal and Schoppmann 2021; Raghuraman and Rindal 2022) have been proposed based on oblivious transfer (OT) extension (Ishai et al. 2003; Kolesnikov et al. 2016) and OPRF (Freedman et al. 2005; Hazay and Lindell 2008). Moreover, since PSI is a special two-party secure computation protocol, some protocols (Huang et al. 2012; Pinkas et al. 2019b, 2015, 2018a) also can be constructed by garbled circuits.

These works are designed for a balanced setting. Below, we introduce the most efficient uPSI protocols (Kales et al. 2019; Resende and Aranha 2021; Chen et al. 2017, 2018; Cong et al. 2021).

OPRF-based uPSI: The OPRF-based uPSI instructs the server to hold the secret key k of PRF and locally apply the PRF to its set to obtain $X' = \{F_k(x), x \in X\}$. After that, it compresses X' by CF and sends the filter to the client. In the online, the client interacts with the server to perform OPRF and gets a pseudorandom set $Y' = \{F_k(y), y \in Y\}$. Finally, the client gets the intersection result by checking whether the items of X' are in CF.

At present, the most efficient protocol is proposed by KRS+19, based on the work of Kiss et al. (2017). They proposed optimized PSI protocols, LowMC-GC-PSI and ECC-NR-PSI. In both of their protocols, the communication and computation costs are linear with the small set ($\mathcal{O}(|Y|)$). But it needs to profile the large set into a cuckoo filter (CF) and send it to the client, which costs $\mathcal{O}(|X|)$ in the offline communication and requires the client to prepare the storage. Recently, Hetz et al. (2023) have made significant improvements in this protocol toward practical application in large-scale sets. The work of Resende and Aranha (2018) also needs offline communication and the same CF as KRS+19. It employs aggressive CF parameters, leading to significantly superior online performance compared to any other protocol, but it suffers from an impractically high false-positive rate of 2^{-13}. Their subsequent work (Resende and Aranha 2021) replaces the CF with a more efficient RSQF for a larger compression rate. However, it leads to a more expensive offline phase.

Most recently, Sun et al. (2024) constructed a maliciously secure uPSI protocol using the Dodis-Yampolskiy PRF (Dodis and Yampolskiy 2005). Their key technique involves encoding the server's set with an oblivious verifiable pseudorandom function (OVRF) and sending it to the client. The client can then execute the PSI protocol with the Server, with complexity linear in the size of the Client's set. The advantage of this approach, compared to directly using OPRF-based protocols, lies in achieving malicious security through verifiability. However, the trade-off is a higher computational and communication cost. Specifically, when the client's set size is 2^{10}, the communication cost of the online phase reaches 63.23 MB, while the work of Cong et al. (2021) only requires 2.04 MB. Moreover, Wang et al. (2025) proposed a highly efficient uPSI protocol based on a novel cryptographic primitive called Client-Independent Relaxed OPRF (CI-rOPRF). Their protocol leverages a VOLE-based construction to enable reusable OPRF encodings that are independent of the client, thereby significantly reducing the online cost of the PSI protocol. However, their efficiency improvement is achieved by relaxing the correctness requirement of the OPRF evaluation. Concretely, their work also use the CF to reduce the communication and takes the same CF parameters as in KRS+19, with FPR of 2^{-29}.

FHE-based uPSI: It is known that FHE can be used to build uPSI (Chen et al. 2017, 2018; Cong et al. 2021; Wu and Yuen 2023). Chen et al. (2017) use FHE to construct the uPSI with communication overhead linear in the smaller set and logarithmic in the larger set. Besides, it utilizes several optimization techniques to reduce computation cost, such as batching, hashing, participating, and windowing. Then, in Chen et al. (2018), they further apply the DH-based OPRF to strengthen the security against a malicious adversary and add efficient support for arbitrary-length items. These works were later refined by Cong et al. (2021), which uses the Paterson-Stockmeyer algorithm and the extremal postage stamp method to reduce the computation and communication costs. Recently, Wu and Yuen (2023) propose the "VBF" and "PoL" techniques upon previous work (Cong et al. 2021) to resolve the long item issue and obtain improved performance. However, it results in increased overhead in the offline phase.

8 Conclusions and Future Work

In this paper, we introduce JAGUAR, a novel PSI protocol based on FHE, incorporating DaC and FV-OPRF techniques. It offers substantial improvements in computational efficiency and communication costs over existing protocols. In addition, the reusable property of the FV-OPRF makes it particularly suitable for the unbalanced case of the client-server scenario. For future work, we plan to achieve full simulation-based security against malicious servers.

Appendix A Functionality

We present the ideal functionality $\mathcal{F}_{\text{MalFixedVOLE}}$ of malicious fixed VOLE in Figure A1.

Functionality $\mathcal{F}_{\text{FixedVOLE}}$

Public Params: Two parties, P_0 and P_1. A finite field $\mathbb{F}$, the length of VOLE tuples d, the number of VOLE instances n.

Functionality: Upon receiving (P_0, **sid**) from P_0 and (P_1, **sid**) from P_1,

1) If the P_1 is honest, sample $[\boldsymbol{b}_i]_{i\in[n]} \leftarrow \mathbb{F}^{d\times n}$ and $[\Delta_i]_{i\in[s]} \leftarrow \mathbb{F}^n$. Otherwise, receive $[\boldsymbol{b}_i]_{i\in[n]} \in \mathbb{F}^{d\times n}$ and $\{\Delta_i\}_{i\in[n]} \in \mathbb{F}^n$ from the adversary.
2) If the P_0 is honest, sample $\boldsymbol{a} \leftarrow \mathbb{F}^d$. Otherwise, receive $\boldsymbol{a}' \in \mathbb{F}^d$ from the adversary and compute $\boldsymbol{c}_i = \Delta_i \cdot \boldsymbol{a} + \boldsymbol{b}_i$ for $i \in [n]$.
3) Send $[\Delta_i]_{i\in[n]}, [\boldsymbol{b}_i]_{i\in[n]}$ to P_1 and $\boldsymbol{a}, [\boldsymbol{c}_i]_{i\in[n]}$ to P_0.

Fig. A1 Ideal functionality $\mathcal{F}_{\text{MalFixedVOLE}}$ of malicious fixed VOLE

Appendix B Security Proofs

B.1 Proof of Theorem 1

We first consider the case of a malicious server and then consider the case of a malicious client. In each case, we construct a PPT simulator *Sim* given access to $\mathcal{F}_{\text{MalFixedVOLE}}$ that runs the PPT adversary $\mathcal{A}$ as a subroutine and emulates the functionality $\mathcal{F}_{\text{MalFixedVOLE}}$.

Malicious Server. *Sim* interact with $\mathcal{A}$ as follows:

1. *Sim* emulates $\mathcal{F}_{\text{MalFixedVOLE}}$ and records the vector $\boldsymbol{a}, \boldsymbol{c}_i, i \in [m]$ that $\mathcal{A}$ sends to $\mathcal{F}_{\text{MalFixedVOLE}}$.
2. *Sim* samples random values Δ_i and computes $\boldsymbol{b}_i = \boldsymbol{c}_i - \Delta_i \cdot \boldsymbol{a}$, $i \in [m]$.
3. *Sim* receives $[\![\chi_1]\!], ..., [\![\chi_d]\!]$ and a^* from $\mathcal{A}$, and computes $[\![b_i^*]\!] = \sum_{j=1}^{d} [\![\chi_j]\!] \cdot \boldsymbol{b}_i[j] + \Delta_i \cdot a^*$. Then *Sim* sends $[\![b_i^*]\!]$ back to $\mathcal{A}$.

Since the Δ_i is randomized uniformly sampled by *sim*, the $\boldsymbol{b}_i = \boldsymbol{c}_i - \Delta_i \cdot \boldsymbol{a}$ is identical to the distribution produced by the real world i-th client. Therefore, the view of the server when interacting with the real-world P_0 and with the simulator *sim* are indistinguishable.

Malicious Client. Here, we first consider the i-th client. *Sim* interact with $\mathcal{A}$ as follows:

1. *Sim* emulates $\mathcal{F}_{\text{MalFixedVOLE}}$ and records the value Δ_i and vector $\boldsymbol{b}_i$ that $\mathcal{A}$ sends to $\mathcal{F}_{\text{MalFixedVOLE}}$.
2. *Sim* samples random number $a^* \in_R \mathbb{F}$, encrypts random number $\{\chi_1, ..., \chi_d\}$, and sends ciphertexts ciphertexts $[\![\chi_1]\!], ..., [\![\chi_d]\!]$ along with a^* to $\mathcal{A}$.
3. Then, *Sim* locally computes $b_i' = \sum_{j=1}^{d} \chi_j \cdot \boldsymbol{b}_i[j] + \Delta_i \cdot a^*$.
4. *Sim* receives $[\![b_i^*]\!]$ from $\mathcal{A}$ and decrypts it to b_i^*.
5. *Sim* compare b_i^* with b_i'. It has three cases:
   1). if the $\mathcal{A}$ honestly follows the protocol and get the value Δ_i and vector $\boldsymbol{b}_i$, then $b_i^* = b_i'$.
   2). if the $b_i^* \neq b_i'$, then *Sim* outputs abort.
   3). if the $\mathcal{A}$ diverge from the protocol but $b_i^* = b_i'$, then *Sim* outputs $fail$.

If the $\mathcal{A}$ samples value $v_1, v_2 \in \mathbb{F}$ and $v_1 + v_2 = b_i'$, the $fail$ happens. Therefore, the probability of $fail$ occurrence is $\frac{1}{|\mathbb{F}|}$. Since we usually at least set $|\mathbb{F}| = 2^{64}$, $fail$ occurs with negligible probability.

Because the FHE scheme is semantically secure, the views of $\mathcal{A}$ when interacting with a real server and with *Sim* are identical. Due to the

failure probability of the consistency check is negligible, the protocol $\Pi_{\text{MalFixedVOLE}}$ of Figure 5 is secure against a malicious client.

B.2 Proof of Theorem 3

We prove Theorem 3 and give the simulator Sim access to $\mathcal{F}_{\text{JAGUAR}}$, which runs the PPT adversary $\mathcal{A}$, as follows.

Malicious Client. Sim interact with $\mathcal{A}$ as follows.

1. In step 3, Sim plays the role of $\mathcal{F}_{\text{FV-OPRF}}$. The simulator observes all the $Y = \{y_i\}_{i\in[n_y]}$ and appends the outputs to the view.
2. In step 7, after receiving ciphertexts from $\mathcal{A}$. Sim uses the random oracle to extract the client's input $Y^* = \{y_i^*\}_{i\in[n_y]}$. Then, Sim computes $Y' = \{y | y \in Y \wedge \nexists\ y' \in Y\ s.t.\ y \neq y' \wedge F(y) = F(y')\}$, and extracts $\hat{Y} = \{y | y \in Y' \wedge\ F(y) \in Y^*\}$.

First, conditioned on there not being any $y \neq y' \wedge F(y) = F(y')$ collisions, it is easy to verify that the simulation above is correct and indistinguishable. Consider some collision s.t. $y \neq y' \wedge F(y) = F(y')$. Note that only one of y, y' being in X, the simulator needs to extract them. The probability of this case is equal to the case $F(y') = F(x), x \in X$. As shown in Rindal and Schoppmann (2021), the probability that the client could find the collision is $\mathcal{O}(2^{-\kappa})$.

Malicious Server. Sim interact with $\mathcal{A}$ as follows.

1. In step 3, Sim emulates $\mathcal{F}_{\text{FV}-\text{OPRF}}$. Sim observes the $(X) = \{x_i\}_{i\in[n_y]}$ and sends OPRF results X' back.
2. Sim forwards X to $\mathcal{F}_{\text{PSI}}$ and gets $Z = X \cap Y$ in response.
3. In step 7, Sim samples random values from $\{0,1\}^{out}\backslash X'$ and encrypts them along with Z to simulate ciphertext results.

This simulation is identical to the real protocol except for the dummy items sampled from $\{0,1\}^{out}\backslash X'$ instead of $\{0,1\}^{out}$. However, since $2^{out} - n_x = \mathcal{O}(2^{\kappa})$, this change is indistinguishable.

Appendix C Extension Performance Comparisons

Note that recent works of Sun et al. (2024) and Wang et al. (2025) also focus on the uPSI and construct efficient protocols. Specifically, Sun et al. (2024) achieves malicious security with two-sided simulation, but at the cost of significantly higher communication and computation overhead. In contrast, Wang et al. (2025) presents a highly efficient uPSI protocol under the semi-honest model. We also compare our protocol with the above two works to show the difference. The results are shown in Table C1.

Comparisons with Sun et al. (2024) The work of Sun et al. (2024) proposed the OVRF and constructed an online efficient uPSI on it. Furthermore, their protocol achieved malicious security at the cost of higher communication overhead. As observed in Table C1, as the client set size $|Y|$ increases, both the communication and computation costs in the online phase of the protocol in Sun et al. (2024) increase accordingly, while the costs in the online phase of OPRF-based protocols are minimally affected by $|X|$. Therefore, their protocol are more suitable for scenarios where the network bandwidth is limited, and the size of $|Y|$ is not too large.

Comparisons with Wang et al. (2025) Similar to KRS+19, the work of *Wanget al.* (2025) is also based on the OPRF paradigm for PSI with CF. The difference is that the protocol of Wang et al. (2025) is based on top-performing PSI, e.g., VOLE-based PSI (Chase and Miao 2020; Rindal and Schoppmann 2021) protocols, where most operations are lightweight, and the communication cost is lower. That is also the reason why their protocol is more efficient than our FHE-based protocol. However, it is important to note that their protocol only supports security in the semi-honest setting for both parties. Moreover, since using the CF to optimize the communication overhead, their protocol also suffers from the PRF problem, which is 2^{-29} in the experiment in Table C1. In contrast, our protocol, JAGUAR, achieves malicious security and guarantees negligible FPR (i.e., $<2^{-\lambda}$).

Table C1 Comparison to the SOTA uPSI (Wang et al. 2025; Sun et al. 2024). All experiments are implemented with single threads. Comm. (MB) includes the client-to-server and server-to-client communication costs.

$\|X\|$	$\|Y\|$	Security	Protocol	Offline		Online	
				Time (s)	Comm. (MB)	Times (s)	Comm. (MB)
2^{20}	5535	Semi-honest	Wang et al. (2025)	2.9	4.19	0.4	0.84
		Malicious	Sun et al. (2024)	196.9	100.6	39.5	225.2
			Ours (DaC+FV)	6.8	5.2	2.6	5.1
	11041	Semi-honest	Wang et al. (2025)	3	4.19	0.5	1.66
		Malicious	Sun et al. (2024)	196.9	100.6	82.3	449.2
			Ours (DaC+FV)	7.9	5.2	4	8.3

References

Baldi P, Baronio R, De Cristofaro E, et al (2011) Countering gattaca: efficient and secure testing of fully-sequenced human genomes. In: Proceedings of the 18th ACM conference on Computer and communications security, pp 691–702

Brakerski Z, Gentry C, Vaikuntanathan V (2014) (leveled) fully homomorphic encryption without bootstrapping. ACM Transactions on Computation Theory (TOCT) 6(3):1–36

Challis MF, Robinson JP (2010) Some extremal postage stamp bases. Journal of Integer Sequences 13(2):3

Chase M, Miao P (2020) Private set intersection in the internet setting from lightweight oblivious prf. In: Advances in Cryptology–CRYPTO 2020: 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17–21, 2020, Proceedings, Part III 40, Springer, pp 34–63

Chen H, Laine K, Rindal P (2017) Fast private set intersection from homomorphic encryption. In: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, pp 1243–1255

Chen H, Huang Z, Laine K, et al (2018) Labeled psi from fully homomorphic encryption with malicious security. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, pp 1223–1237

Chen Y, Wu A, Yang Y, et al (2024) Efficient verifiable cloud-assisted psi cardinality for privacy-preserving contact tracing. IEEE Transactions on Cloud Computing

Cong K, Moreno RC, da Gama MB, et al (2021) Labeled psi from homomorphic encryption with reduced computation and communication. Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security URL https://api.semanticscholar.org/CorpusID:237425502

Diffie W, Hellman M (1976) New directions in cryptography. IEEE Transactions on Information Theory 22(6):644–654. https://doi.org/10.1109/TIT.1976.1055638

Dodis Y, Yampolskiy A (2005) A verifiable random function with short proofs and keys. In: International Workshop on Public Key Cryptography, Springer, pp 416–431

Fan J, Vercauteren F (2012) Somewhat practical fully homomorphic encryption. Cryptology ePrint Archive

Freedman MJ, Nissim K, Pinkas B (2004) Efficient private matching and set intersection. In: International conference on the theory and applications of cryptographic techniques, Springer, pp 1–19

Freedman MJ, Ishai Y, Pinkas B, et al (2005) Keyword search and oblivious pseudorandom functions. In: Theory of Cryptography: Second Theory of Cryptography Conference, TCC 2005, Cambridge, MA, USA, February 10-12, 2005. Proceedings 2, Springer, pp 303–324

Gentry C (2009) Fully homomorphic encryption using ideal lattices. In: Proceedings of the forty-first annual ACM symposium on Theory of computing, pp 169–178

Ghosh S, Simkin M (2019) The communication complexity of threshold private set intersection. In: Annual International Cryptology Conference, Springer, pp 3–29

Hazay C, Lindell Y (2008) Efficient protocols for set intersection and pattern matching with security against malicious and covert adversaries. In: Theory of Cryptography Conference, Springer, pp 155–175

He Y, Tan X, Ni J, et al (2022) Differentially private set intersection for asymmetrical id alignment. IEEE Transactions on Information Forensics and Security 17:3479–3494

Hetz L, Schneider T, Weinert C (2023) Scaling mobile private contact discovery to billions of users. Cryptology ePrint Archive

Huang Y, Evans D, Katz J (2012) Private set intersection: Are garbled circuits better than custom protocols? In: NDSS

Huberman BA, Franklin M, Hogg T (1999) Enhancing privacy and trust in electronic communities. In: Proceedings of the 1st ACM conference on Electronic commerce, pp 78–86

Ishai Y, Kilian J, Nissim K, et al (2003) Extending oblivious transfers efficiently. In: Annual International Cryptology Conference, Springer, pp 145–161

Jing W, Chen X, Chen X, et al (2025) VCR: Fast private set intersection with improved VOLE and CRT-batching. Cryptology ePrint Archive, Paper 2025/1114, URL https://eprint.iacr.org/2025/1114

Kales D, Rechberger C, Schneider T, et al (2019) Mobile private contact discovery at scale. In: 28th USENIX Security Symposium (USENIX Security 19), pp 1447–1464

Kerschbaum F, Blass EO, Mahdavi RA (2023) Faster secure comparisons with offline phase for efficient private set intersection. In: In 30th Annual Network and Distributed System Security Symposium, NDSS 2023, San Diego, California, USA, February 27- March 3, 2023.

Kiss Á, Liu J, Schneider T, et al (2017) Private set intersection for unequal set sizes with mobile applications. Proc Priv Enhancing Technol 2017(4):177–197

Kissner L, Song D (2004) Private and threshold set-intersection. School of Computer Science, Carnegie Mellon University

Kolesnikov V, Kumaresan R, Rosulek M, et al (2016) Efficient batched oblivious prf with applications to private set intersection. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, pp 818–829

Lyubashevsky V, Peikert C, Regev O (2010) On ideal lattices and learning with errors over rings. In: Annual international conference on the theory and applications of cryptographic techniques, Springer, pp 1–23

Meadows C (1986) A more efficient cryptographic matchmaking protocol for use in the absence of a continuously available third party. In: 1986 IEEE Symposium on Security and Privacy, IEEE, pp 134–134

Pagh R, Rodler FF (2001) Cuckoo hashing. In: European Symposium on Algorithms, Springer, pp 121–133

Paterson MS, Stockmeyer LJ (1973) On the number of nonscalar multiplications necessary to evaluate polynomials. SIAM Journal on Computing 2(1):60–66

Pinkas B, Schneider T, Zohner M (2014) Faster private set intersection based on {OT} extension. In: 23rd USENIX Security Symposium (USENIX Security 14), pp 797–812

Pinkas B, Schneider T, Segev G, et al (2015) Phasing: Private set intersection using permutation-based hashing. In: 24th USENIX Security Symposium (USENIX Security 15), pp 515–530

Pinkas B, Schneider T, Weinert C, et al (2018a) Efficient circuit-based psi via cuckoo hashing. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques, Springer, pp 125–157

Pinkas B, Schneider T, Zohner M (2018b) Scalable private set intersection based on ot extension. ACM Transactions on Privacy and Security (TOPS) 21(2):1–35

Pinkas B, Rosulek M, Trieu N, et al (2019a) Spot-light: lightweight private set intersection from sparse ot extension. In: Advances in Cryptology–CRYPTO 2019: 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2019, Proceedings, Part III 39, Springer, pp 401–431

Pinkas B, Schneider T, Tkachenko O, et al (2019b) Efficient circuit-based psi with linear communication. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques, Springer, pp 122–153

Pinkas B, Rosulek M, Trieu N, et al (2020) Psi from paxos: fast, malicious private set intersection. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques, Springer, pp 739–767

Raghuraman S, Rindal P (2022) Blazing fast psi from improved okvs and subfield vole. Cryptology ePrint Archive, Paper 2022/320, URL https://eprint.iacr.org/2022/320, https://eprint.iacr.org/2022/320

Regev O (2009) On lattices, learning with errors, random linear codes, and cryptography. Journal of the ACM (JACM) 56(6):1–40

Resende ACD, Aranha DF (2018) Faster unbalanced private set intersection. In: Financial Cryptography and Data Security: 22nd International Conference, FC 2018, Nieuwpoort, Curaçao, February 26–March 2, 2018, Revised Selected Papers 22, Springer, pp 203–221

Resende ACD, Aranha DF (2021) Faster unbalanced private set intersection in the semi-honest setting. Journal of Cryptographic Engineering 11:21–38

Rindal P, Schoppmann P (2021) Vole-psi: fast oprf and circuit-psi from vector-ole. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques, Springer, pp 901–930

Schoppmann P, Gascón A, Reichert L, et al (2019) Distributed vector-ole: Improved constructions and implementation. In: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, pp 1055–1072

SEAL (2022) Microsoft SEAL (release 4.0). https://github.com/Microsoft/SEAL, microsoft Research, Redmond, WA.

Sun Y, Katz J, Raykova M, et al (2024) Actively secure private set intersection in the client-server setting. In: Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, pp 1478–1492

Wang X, Lu Z, Liang B, et al (2025) Unbalanced psi from client-independent relaxed oblivious prf. Proceedings on Privacy Enhancing Technologies

Weng C, Yang K, Katz J, et al (2021) Wolverine: fast, scalable, and communication-efficient zero-knowledge proofs for boolean and arithmetic circuits. In: 2021 IEEE Symposium on Security and Privacy (SP), IEEE, pp 1074–1091

Wu M, Yuen TH (2023) Efficient unbalanced private set intersection cardinality and user-friendly privacy-preserving contact tracing. In: 32nd USENIX Security Symposium (USENIX Security 23), pp 283–300